eBPFを用いたランサムウェアのファイル操作系列分析

樋口 光輔[1,a)]　小林 良太郎[1,b)]

概要：ランサムウェア被害を抑制するためには，暗号化処理の前後でどのファイル操作がどの順番で発行されるかを把握することが重要である．既存研究である Real-time Open File Backup System (ROFBS) では，file-open イベントを契機として変更前ファイルをバックアップすることで，機械学習ベースの検知モデルがランサムウェアを検知するまでの遅延中に発生する暗号化被害を抑制してきた．一方で，ランサムウェアが実際に open, read, write, create, rename, unlink をどの順番で呼び出すかはファミリごとに異なる可能性がある．たとえば，既存ファイルを open して read した後に write で上書きする挙動，暗号化後に rename を行う挙動，新規ファイルを create して暗号化内容を書き込み，元ファイルを unlink する挙動などが考えられる．このような操作系列の違いは，バックアップや検知に適した観測点を選択するうえで重要な要素となる．本研究では，XFS 上でランサムウェアのファイル操作系列を extended Berkeley Packet Filter (eBPF) により測定する．具体的には，xfs_file_open, xfs_file_read_iter, xfs_file_write_iter, xfs_vn_create, xfs_vn_rename, xfs_vn_unlink を観測し，ファイルごとに open, read, write, create, rename, unlink の観測順序を復元する．ランサムウェアファミリごとのファイル操作系列の違いを明らかにし，リアルタイムバックアップ機構やランサムウェア検知機構における観測点選択の基礎的知見を提供する．

キーワード：ランサムウェア，ファイル操作系列，eBPF

An eBPF-Based Analysis of Ransomware File Operation Sequences

Kosuke Higuchi[1,a)]　Ryotaro Kobayashi[1,b)]

Abstract: To mitigate ransomware damage, it is important to understand which file operations are issued before and after encryption, and in what order they occur. In previous work, the Real-time Open File Backup System (ROFBS) mitigated encryption damage during the delay before a machine-learning-based detection model detects ransomware by creating backups of unmodified files upon file-open events. However, the actual order in which ransomware invokes file operations such as open, read, write, create, rename, and unlink may differ across ransomware families. For example, ransomware may open and read an existing file before overwriting it with encrypted data, rename a file after encryption, or create a new file, write encrypted contents to it, and then unlink the original file. Such differences in file operation sequences are important factors when selecting suitable observation points for backup and detection mechanisms. In this study, we use the extended Berkeley Packet Filter (eBPF) to measure ransomware file operation sequences on XFS. Specifically, we observe xfs_file_open, xfs_file_read_iter, xfs_file_write_iter, xfs_vn_create, xfs_vn_rename, and xfs_vn_unlink and reconstruct the observed order of open, read, write, create, rename, and unlink operations for each file. By identifying differences in file operation sequences among ransomware families, this study provides fundamental insights for selecting observation points in real-time backup and ransomware detection mechanisms.

Keywords: Ransomware, File Operation Sequences, eBPF

1. はじめに

暗号化型ランサムウェアは，端末やサーバに保存された

[1] 工学院大学
Kogakuin University, Shinjuku, Tokyo 163-8677, Japan
[a)] ed25001@g.kogakuin.jp
[b)] ryo.kobayashi@cc.kogakuin.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ファイルを暗号化し，その復号と引き換えに金銭を要求するマルウェアである．ランサムウェアによる被害を抑制するためには，攻撃を早期に検知するだけでなく，検知までに暗号化されるファイルを保護し，復元可能性を確保することが重要である．しかし，機械学習などを用いたランサムウェア検知では，攻撃開始から判定までに一定の時間を要する．この検知遅延中にもファイル暗号化は進行するため，検知のみでは暗号化被害を完全に防止できない．

この問題に対し，我々は，ファイルの open 操作を契機として変更前ファイルのバックアップを作成する Real-time Open File Backup System (ROFBS) を提案した [1]．ROFBS は，機械学習ベースの検知モデルがランサムウェアを検知するまでの間に，file-open イベントを契機として暗号化対象となる可能性があるファイルバックアップをすることで，検知遅延中の被害を抑制する．open 操作は，通常，ファイル内容の read や write より前に発生するため，暗号化前のファイルをバックアップするための観測点として利用できる．

一方，ランサムウェアによる暗号化処理は，単一のファイル操作系列によって実行されるとは限らない．例えば，既存ファイルを open および read した後，暗号化内容を同一ファイルへ write する処理が考えられる．また，元ファイルを read した後に新規ファイルを create し，暗号化内容を新規ファイルへ write した後，元ファイルを unlink する，または暗号化済みファイルを rename して元ファイルと置き換える処理も考えられる．さらに，ファイルを read せずに上書きする処理や，多数のファイルを open または read した後，その一部のみを暗号化する処理も存在する可能性がある．

このようなファイル操作系列の違いは，リアルタイムバックアップ機構やランサムウェア検知機構における観測点を選択するうえで重要である．open を観測すれば，暗号化処理より早い段階で対象ファイルを把握できる可能性がある一方，最終的に暗号化されないファイルも観測する可能性がある．write はファイルに対する書き込みの発生を捉えられるが，ファイル内容の変更が開始される時点に近いため，変更前ファイルを保存するための観測点としては遅い可能性がある．rename や unlink は，暗号化後のファイル置換や削除を捉えられる可能性があるが，その時点では元ファイルの内容が既に変更または削除されている可能性がある．したがって，適切な観測点を選択するためには，各操作の発生回数だけでなく，ファイル単位で open, read, write, create, rename, および unlink がどのような順序で発行されるかを把握する必要がある．

そこで本研究では，XFS 上で動作するランサムウェアが発行するファイル操作を extended Berkeley Packet Filter (eBPF) により観測し，ファイルごとの操作系列を復元する．具体的には，open, read, write, create, rename, および unlink に対応する XFS のカーネル関数を観測する．取得したイベントを対象ファイルごとに分類し，観測時刻順に並べることで，ファイル操作系列を構成する．

本論文の構成は以下の通りである．第 2 章では関連研究について述べる．第 3 章ではファイル操作系列の観測手法を説明し，第 4 章では実験環境を示す．第 5 章では実験結果を示し，第 6 章では観測結果について考察する．第 7 章では本研究をまとめ，今後の課題を述べる．

2. 関連研究

Ju らは，ランサムウェアのファイル Input/Output (I/O) 操作系列を意図的に変化させ，既存の振る舞いベース検知を回避する手法を提案している [2]．実験の結果，I/O 系列パターンの変化によって既存の検知手法や商用アンチランサムウェア製品を回避できることが示され，未知の I/O パターンにも対応可能な検知手法の必要性が明らかになった．Zhao らは，回避型ランサムウェアに特有の反復的な I/O 挙動の相関と，暗号化ファイルのバイト列分布を用いた細粒度コンテンツ解析を組み合わせる適応型検知システム ERW-Radar を提案している [3]．これにより，回避型ランサムウェアに対して 96.18%の検知精度，5.36%の偽陽性率を達成しつつ，Central Processing Unit (CPU) 使用率およびメモリ使用率の平均オーバーヘッドをそれぞれ 5.09%, 3.80%に抑えられることが示された．Hitaj らは，プロセス単位の挙動ではなくファイル単位のアクセス挙動を特徴量として利用することで，回避攻撃に対して頑健なランサムウェア検知手法「Minerva」を提案している [4]．評価の結果，未知のランサムウェアや検知回避を目的とした亜種に対しても高い検知性能を維持し，検知されたランサムウェアの 99%以上を活動開始から 0.52 秒以内に検出可能であることが示された．Guo らは，ランサムウェアによる暗号化処理の計算負荷とファイル I/O 量の時間的相関に着目した検知手法 RansomRadar を提案している [5]．411 個・89 ファミリのランサムウェアを用いた評価により，検知率 100%，偽陽性率 2.68%，F1 スコア 96.03%を達成し，ステルスな暗号化や正常 I/O を模倣する回避型ランサムウェアに対しても有効であることが示された．van Sloun らは，プロセスの挙動に応じて監視する I/O 特徴量の数を動的に変更する多段階型ランサムウェア検知システムを提案している [6]．これにより，従来の単一段階型手法と同程度の検知精度を維持しながら，I/O 監視によるオーバーヘッドを約 1 桁削減でき，リアルタイムなランサムウェア検知が実用的に可能になることが示された．Brodzik らは，Linux 環境において eBPF を用いてシステムコール情報を収集し，決定木および多層パーセプトロンによる機械学習推論をカーネル空間内で実行するランサムウェア検知手法を提案している [7]．これにより，いずれのモデルでも macro F1 スコア 95%以上を維持しつつ，ユーザ空間での推論と

© 2026 Information Processing Society of Japan

比較して大幅に処理遅延を削減でき，リアルタイムなランサムウェア検知に有効であることが示された．Wang らは，大規模な産業環境での運用を想定し，監視層で疑わしいプロセスを段階的に絞り込んだ後，詳細な振る舞い解析を行うリアルタイムかつ軽量なランサムウェア検知・対応システム CanCal を提案している [8]．1,761 個のランサムウェアと約 3,000,000 件のイベントを含む 5 か月間の評価により，既存の最先端手法と同等の検知性能を維持しながら，30 ms 以内の推論と最大 3 秒以内のリアルタイム対応を実現できることが示された．Chew らは，Android 上のクリプトランサムウェアが示すシステムコールレベルの共通した振る舞いパターンを 2 層の Finite State Machine (FSM) でモデル化し，ストリーミング処理によってリアルタイムに検知する手法を提案している [9]．6 種類のランサムウェアファミリと 500 個の良性アプリケーションを用いたストリーミング方式の評価では，真陽性率 100%を維持しつつ，偽陽性率を 0.4%まで低減できることが示された．von der Assen らは，Linux 環境において，ファイルアクセス前の情報を取得するオーバレイファイルシステムと機械学習による検知を統合したランサムウェア対策手法 GuardFS を提案している [10]．検知後にファイルアクセスの難読化・遅延・追跡を行う 3 種類の防御構成を評価し，計算資源と利便性のトレードオフはあるものの，ランサムウェアによるデータ損失を大幅に削減できることが示された．

3. ファイル操作系列の観測手法

本研究では，暗号化処理中に XFS 上で発生するファイル操作を，eBPF および BPF Compiler Collection (BCC) を用いて観測する．eBPF プログラムを XFS のカーネル関数へアタッチし，カーネル空間で発生したファイル操作を取得する．取得したイベントは BCC を介してユーザ空間へ送信し，対象ファイルごとに観測時刻順へ並べることでファイル操作系列を復元する．

ランサムウェアによるファイル暗号化では，複数のファイル操作系列が考えられる．

図 1 に，元ファイルへ暗号化内容を書き戻す場合に考えられるファイル操作系列の一例を示す．この図は，ランサムウェアを特定の方式へ分類するためのものではなく，open, read, write, および rename の相対的な発生位置を説明するための概念図である．

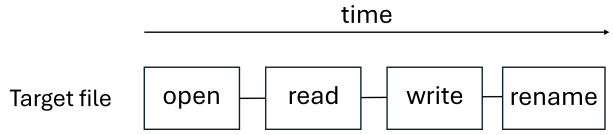


図 1 元ファイルへ暗号化内容を書き戻す場合に考えられるファイル操作系列の例

図 1 に示す例では，元ファイルに対して open および read を実行した後，暗号化内容を同一ファイルへ write する．暗号化後にファイル名や拡張子を変更する場合には，write の後に rename が発生する可能性がある．一方，元ファイルの内容を読み込まずに上書きする場合や，暗号化後にファイル名を変更しない場合には，read または rename が発生しない可能性もある．

図 2 に，元ファイルとは異なるファイルへ暗号化内容を書き込む場合に考えられるファイル操作系列の一例を示す．

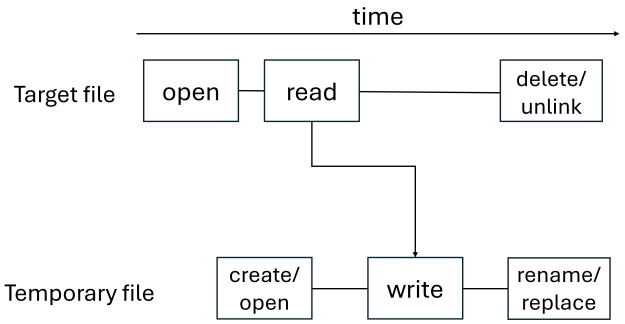


図 2 元ファイルとは異なるファイルへ暗号化内容を書き込む場合に考えられるファイル操作系列の例

図 2 に示す例では，元ファイルに対して open および read を実行した後，新たなファイルを create および open し，そのファイルへ暗号化内容を write する．その後，元ファイルに対する unlink や，新規ファイルを元ファイルと置き換えるための rename が発生する可能性がある．この場合，最初の open および read の対象となる元ファイルと，create 以降の open および write の対象となる新規ファイルは異なる．また，元ファイルの削除と新規ファイルの rename の一方のみが実行される場合も考えられる．図中の create/open は新規ファイルに対する create と open，delete/unlink は unlink による元ファイルの削除，rename/replace は rename によるファイルの置換を表す．delete および replace は独立した観測対象の操作名ではなく，それぞれ unlink および rename の結果を説明する表記である．

本研究では，open, read, write, create, rename, および unlink の 6 種類を観測対象とする．表 1 に，各操作の観測に使用する XFS カーネル関数を示す．open 操作には xfs_file_open, read 操作には xfs_file_read_iter, write 操作には xfs_file_write_iter を使用する．また，create 操作には xfs_vn_create, rename 操作には xfs_vn_rename, unlink 操作には xfs_vn_unlink を使用する．

© 2026 Information Processing Society of Japan

表 1 ファイル操作系列の観測に使用する XFS カーネル関数

Operation	Kernel Function
Open	xfs_file_open
Read	xfs_file_read_iter
Write	xfs_file_write_iter
Create	xfs_vn_create
Rename	xfs_vn_rename
Unlink	xfs_vn_unlink

本研究で扱う各操作は，対応する XFS カーネル関数の実行を捉えたイベントを意味する．本実験で取得したイベントログからは各操作の発生と順序を確認できるが，read で読み出された内容，write で書き込まれた内容，および各操作の目的は取得していない．したがって，write イベントのみから暗号化内容の書き込みか補助的な書き込みかを判定することや，rename または unlink イベントのみから暗号化処理の段階を判定することはできない．

取得したイベントは，ユーザ空間において観測時刻順に整列する．デバイス番号および inode 番号の組をファイル識別子とし，ファイル単位の操作系列としてまとめる．inode 番号を取得できないイベントについては，デバイス番号とファイル名の組を代替の識別子とする．例えば，同一ファイルに対して open, read, read, write, write, rename の順にイベントが観測された場合，次の操作系列として記録する．

open → read → read → write → write → rename

検体間で操作系列を比較しやすくするため，連続して発生した同一種類の操作を 1 つにまとめた正規化系列も生成する．前述の系列は，正規化後には次のように表される．

open → read → write → rename

圧縮前の個々のイベントは，観測時刻とともにイベントログに保持する．

観測された操作系列の違いは，リアルタイムバックアップ機構やランサムウェア検知機構における観測点を選択するうえで重要である．例えば，多くの暗号化対象ファイルで open が write より前に観測される場合，open は最初の write より前にバックアップを開始するための候補観測点となる．本研究では，観測された操作系列に基づき，単一の操作をあらかじめ最適な観測点として仮定するのではなく，バックアップおよび検知に必要な観測点について考察する．

4. 実験環境

表 2 に，本研究で使用した実験環境を示す．ホストマシン上に仮想マシンを構築し，仮想マシン上でランサムウェア検体を実行した．ホストの Operating System (OS) には Windows 11 を使用し，仮想マシンの OS には AlmaLinux 10.1 を使用した．仮想マシンには 4 コアの仮想プロセッサ，16 GB のメモリ，および 100 GB のストレージを割り当てた．また，仮想マシン上のファイルシステムには XFS を使用した．

表 2 Experimental Environment (Host and Virtual Machine)

	Host Machine	Virtual Machine
CPU	Intel Core i7-12650H	4 Cores
Memory	32 GB (DDR4 2,400 MT/s)	16 GB
Storage	512 GB (M.2 NVMe)	100 GB
OS	Windows 11	AlmaLinux 10.1

本研究では，ランサムウェア検体ごとに観測されるファイル操作系列を比較するため，複数のランサムウェア検体を用いて実験を行った．評価対象は，HelloKitty, AvosLocker, Conti, および Sodinokibi の 4 検体である．Sodinokibi は REvil の名称でも知られているが，本論文では検体収集時に付与されていた名称に従い，Sodinokibi と表記する．本研究で使用した検体は，2023 年 5 月から 10 月までの間に取得した．表 3 に，実験で使用した検体と Message Digest Algorithm 5 (MD5) ハッシュ値を示す．

表 3 実験で使用したランサムウェア検体

Ransomware	MD5
HelloKitty	f597d74b84929ded948fc42c6f6210e5
AvosLocker	d3cafcd46dea26c39dec17ca132e5138
Conti	cfb6d21ffe7c4279f761f2351c0810ee
Sodinokibi	c83df66c46bcbc05cd987661882ff061

ファイル操作系列の観測対象には，ランサムウェアによる暗号化挙動を観測するため，/home/user/victim/以下に配置した実験用ファイル群を用いた．実験用ファイルは 303 個であり，jpeg, txt, および docx の 3 種類のファイルをそれぞれ 101 個ずつ配置した．各ランサムウェア検体の実行では，これら 303 個の実験用ファイルについてファイル操作イベントを取得し，ファイル単位の操作系列を復元した．観測対象は，評価対象のランサムウェアプロセスが/home/user/victim/以下のファイルに対して発行した操作に限定した．

5. 実験結果

本節では，評価対象のランサムウェアプロセスが実験用ファイルに対して発行したファイル操作の観測結果を示す．

5.1 観測されたファイル操作数

表 4 に，303 個の実験用ファイルに対して観測された操作数を示す．

© 2026 Information Processing Society of Japan

表 4 実験用ファイルに対して観測されたファイル操作数

Ransomware	Open	Read	Write	Rename
AvosLocker	303	231	575	303
Conti	303	307	610	303
HelloKitty	606	802	1,714	606
Sodinokibi	303	2,414	1,090	303

AvosLocker, Conti, および Sodinokibi では，各実験用ファイルに対して open と rename が 1 回ずつ観測された．HelloKitty では，各実験用ファイルに対して open と rename が 2 回ずつ観測された．read の観測数は，AvosLocker で 231 回，Conti で 307 回，HelloKitty で 802 回，Sodinokibi で 2,414 回であった．write の観測数は，AvosLocker で 575 回，Conti で 610 回，HelloKitty で 1,714 回，Sodinokibi で 1,090 回であった．

5.2 正規化操作系列

表 5 に，各検体で観測された主な正規化操作系列を示す．表中では，各操作を open, read, write, rename と省略せずに表記する．同一の read と write の組が繰り返された場合は，括弧右上の数値によって繰り返し回数を示す．

AvosLocker では，Open → Write → Rename が 174 ファイルで観測され，303 ファイル中 57.43%を占めた．次いで，Open → Read → Write → Rename が 72 ファイルで観測され，23.76%を占めた．Open → Read → Write → Read → Write → Rename は 30 ファイル，Open → Write → Read → Write → Rename は 27 ファイルで観測された．

Conti では，302 ファイルで Open → Write → Read → Write → Rename が観測され，303 ファイル中 99.67%を占めた．残る 1 ファイルでは，open の後に write が発生し，read と write が 5 回繰り返された後に rename が観測された．

HelloKitty では，すべての実験用ファイルで Open → Rename → Open から始まる系列が観測された．最も多かった系列は，Open → Rename → Open → Write → $(\text{Read} \rightarrow \text{Write})^2$ → Rename であり，188 ファイル，62.05%を占めた．次いで，read と write の組を 4 回繰り返す系列が 72 ファイル，3 回繰り返す系列が 35 ファイルで観測された．その他の系列は 8 ファイルで観測された．

Sodinokibi では，すべての実験用ファイルで open の後に read が観測された．最も多かった系列は，Open → $(\text{Read} \rightarrow \text{Write})^2$ → Rename であり，191 ファイル，63.04%を占めた．次いで，read と write の組を 4 回繰り返す系列が 79 ファイル，3 回繰り返す系列が 28 ファイルで観測された．その他の系列は 5 ファイルで観測された．

5.3 最初の Write より前に観測された操作

表 6 に，各実験用ファイルの最初の write より前に観測された操作を示す．括弧内は 303 ファイルに対する割合を百分率で表す．

表 6 最初の Write より前に各操作が観測されたファイル数

Ransomware	Open	Read	Rename
AvosLocker	303 (100.00%)	102 (33.66%)	0 (0.00%)
Conti	303 (100.00%)	0 (0.00%)	0 (0.00%)
HelloKitty	303 (100.00%)	1 (0.33%)	303 (100.00%)
Sodinokibi	303 (100.00%)	303 (100.00%)	0 (0.00%)

すべての検体において，303 個の実験用ファイルすべてで open が最初の write より前に観測された．read が最初の write より前に観測されたファイル数は，AvosLocker で 102 ファイル，Conti で 0 ファイル，HelloKitty で 1 ファイル，Sodinokibi で 303 ファイルであった．HelloKitty では，303 個の実験用ファイルすべてで最初の write より前に rename が観測された．AvosLocker, Conti, および Sodinokibi では，最初の write より前に rename が観測されたファイルは存在しなかった．

5.4 Create，Rename，および Unlink の観測結果

表 7 に，create, rename, および unlink の観測結果と，観測されたファイル名の例を示す．暗号化対象となった 303 個の実験用ファイルについては，いずれの検体でも create は操作系列に含まれなかった．一方，create イベントは，ランサムノートやロックファイルなどの補助ファイルを作成する際に観測された．ここで，create における Observed Count は create イベントの観測回数を表し，rename における Observed Count は，rename が観測された実験用ファイル数を表す．

AvosLocker では README_FOR_RESTORE，Conti では readme.txt の作成時に create イベントが観測された．HelloKitty では，元のファイル名に README_TO_RESTORE を付加した補助ファイルが作成された．Sodinokibi では，元のファイル名に lck を付加したロックファイルと，vemar-readme.txt が作成された．create イベントの観測回数は，AvosLocker で 1 回，Conti で 4 回，HelloKitty で 306 回，Sodinokibi ではロックファイルについて 306 回，vemar-readme.txt について 4 回であった．

rename は，すべての検体で 303 個の実験用ファイルすべてに対して観測された．AvosLocker では元のファイル名に avoslinux，Conti では conti，Sodinokibi では vemar を付加する rename が観測された．HelloKitty では，各実験用ファイルに対して 2 回の rename が観測された．1 回目の rename では元のファイル名に tmp_が付加され，2 回目の rename では tmp_を含むファイル名から crypt を付加したファイル名へ変更された．

本実験で 4 検体の実行時に取得したイベントログにおけ

© 2026 Information Processing Society of Japan

表 5 実験用ファイルで観測された主な正規化操作系列

Ransomware	Normalized Sequence	Files	Ratio [%]
AvosLocker	Open → Write → Rename	174	57.43
AvosLocker	Open → Read → Write → Rename	72	23.76
AvosLocker	Open → Read → Write → Read → Write → Rename	30	9.90
AvosLocker	Open → Write → Read → Write → Rename	27	8.91
Conti	Open → Write → Read → Write → Rename	302	99.67
Conti	Open → Write → (Read → Write)^5 → Rename	1	0.33
HelloKitty	Open → Rename → Open → Write → (Read → Write)^2 → Rename	188	62.05
HelloKitty	Open → Rename → Open → Write → (Read → Write)^4 → Rename	72	23.76
HelloKitty	Open → Rename → Open → Write → (Read → Write)^3 → Rename	35	11.55
HelloKitty	Other	8	2.64
Sodinokibi	Open → (Read → Write)^2 → Rename	191	63.04
Sodinokibi	Open → (Read → Write)^4 → Rename	79	26.07
Sodinokibi	Open → (Read → Write)^3 → Rename	28	9.24
Sodinokibi	Other	5	1.65

表 7 Create，rename，および unlink の観測結果とファイル名の例

Operation	Ransomware	Observed Count	Example File Name or Rename
Create	AvosLocker	1 event	README_FOR_RESTORE
Create	Conti	4 events	readme.txt
Create	HelloKitty	306 events	0011-txt.txt.README_TO_RESTORE
Create	Sodinokibi	306 events	0012-txt.txt.lck
Create	Sodinokibi	4 events	vemar-readme.txt
Rename	AvosLocker	303 files	0004-docx.docx → 0004-docx.docx.avoslinux
Rename	Conti	303 files	0011-txt.txt → 0011-txt.txt.conti
Rename	HelloKitty	303 files, twice per file	0012-txt.txt → 0012-txt.txt.tmp_ → 0012-txt.txt.crypt
Rename	Sodinokibi	303 files	0014-txt.txt → 0014-txt.txt.vemar
Unlink	All samples	0 events	–

る unlink イベントの観測回数は 0 回であった．

6. 考察

6.1 ランサムウェア検体間における操作系列の違い

ファイル操作数と正規化操作系列は，評価した検体間で大きく異なった．表 4 に示したように，AvosLocker の read と write の観測数はそれぞれ 231 回と 575 回であり，Conti では 307 回と 610 回であった．HelloKitty では open と rename がそれぞれ 606 回観測され，いずれも他の 3 検体の 2 倍であり，read は 802 回，write は 1,714 回であった．また，Sodinokibi では read が 2,414 回，write が 1,090 回観測され，read の観測数は 4 検体の中で最も多かった．これらの操作数は，各検体で同じ操作が発生する回数に違いがあることを示すが，操作数だけでは操作間の順序を判別できないため，正規化操作系列と併せて解釈する必要がある．

表 5 に示したように，AvosLocker では，read を含まない Open → Write → Rename が 57.43%と最も多かった一方，その他には write より前または後に read を含む系列が観測された．この結果から，今回評価した AvosLocker 検体では，すべてのファイルに対して同一の read と write の順序が用いられていたわけではないことがわかる．

Conti では，303 ファイル中 302 ファイル，すなわち 99.67%で Open → Write → Read → Write → Rename が観測された．したがって，今回評価した Conti 検体では，open の後に最初の write が発生し，その後に read と write が発生する系列がほぼ共通していた．

HelloKitty では，すべての対象ファイルで Open → Rename → Open から始まる系列が観測された．そのうち 62.05%では read と write の組が 2 回繰り返され，また，繰り返し回数が異なる系列が観測された．最初の rename は最初の write より前に発生しており，元ファイル名に tmp_ を付加した後，同じ inode に対して open および write が発生していた．この結果から，rename は必ずしも write 後の操作ではなく，write より前のファイル処理に使用される場合があることがわかる．また，一連の read および write の後には，tmp_から crypt を付加したファイル名への 2 回目の rename が観測された．

Sodinokibi では，すべての対象ファイルで open の後に read が発生し，read と write の組が複数回繰り返された後に rename が観測された．最も多い系列は全体の 63.04%であり，その他には read と write の繰り返し回数が異なる系

© 2026 Information Processing Society of Japan

列が観測された．この結果から，今回評価した Sodinokibi 検体では，read と write を交互に実行する系列が主要な処理であったことがわかる．

以上の結果から，open, read, write, および rename の相対的な順序は検体間で異なり，単一の代表系列だけで評価対象の挙動を説明することは困難であることがわかった．

6.2 最初の Write より前に観測される操作

表 6 に示したように，すべての検体において，303 個の実験用ファイルすべてで open が最初の write より前に観測された．この結果から，今回評価した 4 検体では，open は write より前に一貫して観測可能な操作であった．したがって，最初の write より前の観測点を必要とするバックアップ機構や検知機構において，open は有力な候補となる．

read が最初の write より前に観測された割合は検体によって異なった．Sodinokibi では 303 ファイルすべて，AvosLocker では 102 ファイルで read が最初の write より前に観測された．一方，Conti では該当ファイルが存在せず，HelloKitty では 1 ファイルのみであった．したがって，read は Sodinokibi のような系列に対しては最初の write より前の観測点となるが，すべての検体に共通する観測点ではない．

HelloKitty では，すべての対象ファイルで rename が最初の write より前に観測された．一方，他の 3 検体では rename は最初の write より後に観測された．この結果は，rename を write 後の処理としてのみ捉えることが適切でない場合があることを示している．rename を検知やバックアップの観測点として利用する場合は，rename が操作系列のどの位置で発生したかを併せて確認する必要がある．

6.3 Create，Rename，および Unlink の観測点としての位置付け

表 7 に示したように，暗号化対象となった 303 個の実験用ファイルについては，いずれの検体でも create は操作系列に含まれなかった．この結果は，少なくとも今回の実験では，新たに作成したファイルへ暗号化内容を書き込む系列が，暗号化対象ファイルについて確認されなかったことを示している．一方，create イベントは，ランサムノートやロックファイルなどの補助ファイルに対して観測された．AvosLocker では README_FOR_RESTORE，Conti では readme.txt, HelloKitty では README_TO_RESTORE を末尾に持つファイル, Sodinokibi では lck を末尾に持つファイルおよび vemar-readme.txt の作成時に create が観測された．

create イベントの観測回数は，AvosLocker で 1 回，Conti で 4 回であったのに対し，HelloKitty では 306 回，Sodinokibi では 2 種類の補助ファイルに対してそれぞれ 306 回および 4 回であり，検体間で大きく異なった．したがって，今回評価した検体では，create は暗号化対象ファイルそのものを早期に特定する観測点ではなく，ランサムノートやロックファイルなど，ランサムウェアが生成する補助ファイルを捉える観測点として利用できる可能性がある．ただし，補助ファイルに対する create の発生頻度は検体によって異なるため，create の観測回数に一律のしきい値を設定する方法では，検体ごとに検知性能が異なる可能性がある．このため，create を検知に利用する場合には，観測回数だけでなく，生成されたファイル名も併せて分析する必要がある．

rename は，すべての検体で 303 個の実験用ファイルすべてに対して観測された．表 7 に示したように，AvosLocker では元のファイル名に avoslinux，Conti では conti，Sodinokibi では vemar を付加する rename が観測された．HelloKitty では，各実験用ファイルについて，元のファイル名に tmp_を付加する rename と，その後に crypt を付加する rename の 2 回が観測された．

rename はファイル名変更や拡張子付与を捉える補助的な観測点として利用できる一方，前節で述べた系列内の発生位置の違いから，イベントの存在のみで暗号化処理の開始または完了を一意に判断することはできない．

本実験で取得したイベントログでは，unlink イベントは観測されなかった．この結果から，本実験の条件下では，unlink をバックアップまたは検知の観測点として利用する可能性について，十分な評価を行うには至らなかった．また，unlink イベントが観測されなかった要因については，本実験で取得したイベントログの範囲から特定することは困難である．したがって，unlink の有用性については今後の検討課題とし，フック対象関数やイベント取得条件，ログ出力の条件などを整理した上で，追加の評価を行う必要がある．

6.4 バックアップおよび検知における観測点選択への示唆

今回評価したすべての検体で，open は最初の write より前に観測された．そのため，単一操作を用いて最初の write より前のファイルアクセスを捉える場合，open は評価した候補の中で最も一貫した観測点であった．

read は，Sodinokibi では全対象ファイルで write より前に観測されたが，Conti および HelloKitty ではほとんど観測されなかった．したがって，read は単独の共通観測点としては不十分である一方，open と組み合わせることで，最初の write より前に read を行うファイル操作系列を識別するための補助的な特徴として利用できる可能性がある．

write は，すべての暗号化対象ファイルに対して観測されたが，ファイル内容の変更開始と同時点に近い操作である．ただし，本実験では書き込み内容を取得していないため，write イベントのみから暗号化内容の書き込みか補助的な書き込みかを判定することはできない．そのため，評

© 2026 Information Processing Society of Japan

価対象プロセスが書き込みを行うファイルを特定する観測点として利用できる一方，変更前ファイルを保存するためのバックアップ契機としては遅い可能性がある．また，Conti のように最初の write の後に read が発生する系列も観測されたため，最初の write だけで暗号化処理全体を表現できるとは限らない．

以上より，バックアップおよび検知の観測点を選択する際には，単一操作の有無だけでなく，read と write の順序および rename の発生位置を考慮する必要がある．操作系列を特徴量として利用することで，単一イベントのみを用いる方式よりも，検体ごとのファイル処理の違いを反映した検知や防御が可能になると考えられる．

7. おわりに

本研究では，ランサムウェアが暗号化処理中に発行するファイル操作を明らかにするため，eBPF および BCC を用いて XFS 上のファイル操作を観測し，ファイル単位の操作系列を復元した．観測対象は open, read, write, create, rename, および unlink であり，取得したイベントを観測時刻順に整列した後，デバイス番号と inode 番号の組を基にファイルごとの系列を構成した．また，連続して発生した同一種類の操作を 1 つにまとめることで，検体間の比較に用いる正規化操作系列を生成した．

AvosLocker, Conti, HelloKitty, および Sodinokibi の 4 検体について，各 303 個の実験用ファイルを分析した結果，観測された主要な操作系列は検体間で異なった．AvosLocker では open, write, rename の順序が最も多く観測され，Conti では open の後に write が発生し，その後に read と write が続く系列がほぼすべてのファイルで観測された．HelloKitty では，最初の write より前に rename が発生し，再度 open した後に read と write を繰り返す系列が観測された．Sodinokibi では，open の後に read と write を複数回繰り返し，最後に rename を行う系列が主要であった．このことから，ランサムウェアの暗号化処理を単一の代表的なファイル操作系列だけで説明することは困難であることがわかった．

最初の write より前に観測された操作を比較した結果，4 検体のすべてにおいて，303 個の実験用ファイルすべてで open が最初の write より前に観測された．一方，read が最初の write より前に観測されたファイルは，Sodinokibi では 303 個すべて，AvosLocker では 102 個，HelloKitty では 1 個であり，Conti では確認されなかった．また，HelloKitty ではすべての対象ファイルで rename が最初の write より前に観測されたのに対し，他の 3 検体では rename は最初の write より後に観測された．したがって，open は今回評価した検体に共通して write より前に観測できる操作であった一方，read および rename の相対的な位置は検体に依存することが確認された．

create イベントは，今回分析した暗号化対象ファイルの操作系列には含まれず，ランサムノートやロックファイルなどの補助ファイルに対して観測された．rename はすべての検体の全対象ファイルで観測されたが，操作系列内で発生する位置は検体によって異なった．また，本実験で取得したイベントログには unlink が記録されていなかったため，unlink を含む操作系列については評価できなかった．これらの結果から，create や rename などの単一イベントの有無だけでは暗号化処理の段階を一意に判断できず，前後の open, read, および write を含む系列として分析する必要がある．

以上より，ランサムウェアのファイル操作系列は検体ごとに異なり，バックアップや検知に適した観測点を単一の操作だけで決定することは困難であることがわかった．open は今回評価したすべての検体で最初の write より前に観測されたため，write より前のファイルアクセスを捉える観測点として有力である．一方，read や rename の有効性は操作系列内の位置に依存するため，単一イベントの有無ではなく，操作の順序を組み合わせて利用することが重要である．

今後は，同一ファミリの異なる検体や他のランサムウェアファミリを追加し，観測される系列の多様性を検証する．さらに，最初に観測された write が暗号化内容の書き込みであるか，ヘッダやマーカーなどの補助的な書き込みであるかを，書き込み位置や内容変化と対応付けて分析する．最終的には，観測されたファイル操作系列を，リアルタイムバックアップの開始条件やランサムウェア検知の特徴量として利用する方式を検討する．

謝辞　本研究の一部は，JSPS 科研費 26K14827 の支援により行った．

参考文献

[1] K. Higuchi and R. Kobayashi, “Real-Time Open-File Backup System with Machine-Learning Detection Model for Ransomware,” International Journal of Information Security, Vol. 24, No. 1, p. 54, 2025.

[2] I.H. Ju and H.K. Kim, “Elastic Shifts: I/O Sequence Patterns of Ransomware and Detection Evasion,” IEEE Access, Vol. 13, pp. 122341–122353, 2025.

[3] L. Zhao, Y. Zhang, Z. Wang, F. Yuan and R. Hou, “ERW-Radar: An Adaptive Detection System against Evasive Ransomware by Contextual Behavior Detection and Fine-Grained Content Analysis,” Proceedings of the 32nd Network and Distributed System Security Symposium, 2025.

[4] D. Hitaj, G. Pagnotta, F. De Gaspari, L. De Carli and L.V. Mancini, “Minerva: A File-Based Ransomware Detector,” Proceedings of the 20th ACM Asia Conference on Computer and Communications Security, pp. 576–590, 2025.

[5] L. Guo, Y. Hou, C. Zhou, Q. Zhang and Y. Jiang, “Ransomware Detection through Temporal Correlation between Encryption and I/O Behavior,” Proceedings of the

© 2026 Information Processing Society of Japan

ACM on Software Engineering, Vol. 2, No. FSE, pp. 197–218, 2025.

[6] C. van Sloun, V. Woeste, K. Wolsing, J. Pennekamp and K. Wehrle, “Detecting Ransomware Despite I/O Overhead: A Practical Multi-Staged Approach,” Proceedings of the 32nd Network and Distributed System Security Symposium, 2025.

[7] A. Brodzik, T. Malec-Kruszyński, W. Niewolski, M. Tkaczyk, K. Bocianiak and S.-Y. Loui, “Ransomware Detection Using Machine Learning in the Linux Kernel,” arXiv preprint arXiv:2409.06452, 2024.

[8] S. Wang, F. Dong, H. Yang, J. Xu and H. Wang, “CanCal: Towards Real-Time and Lightweight Ransomware Detection and Response in Industrial Environments,” Proceedings of the 31st ACM SIGSAC Conference on Computer and Communications Security, pp. 2326–2340, 2024.

[9] C.J.W. Chew, V. Kumar, P. Patros and R. Malik, “Real-Time System Call-Based Ransomware Detection,” International Journal of Information Security, Vol. 23, No. 3, pp. 1839–1858, 2024.

[10] J. von der Assen, C. Feng, A. Huertas Celdrán, R. Oleš, G. Bovet and B. Stiller, “GuardFS: A File System for Integrated Detection and Mitigation of Linux-Based Ransomware,” Journal of Information Security and Applications, Vol. 93, p. 104078, 2025.

© 2026 Information Processing Society of Japan